ML Ops Group Assignment - Final Report
Group # 37

Group Members:

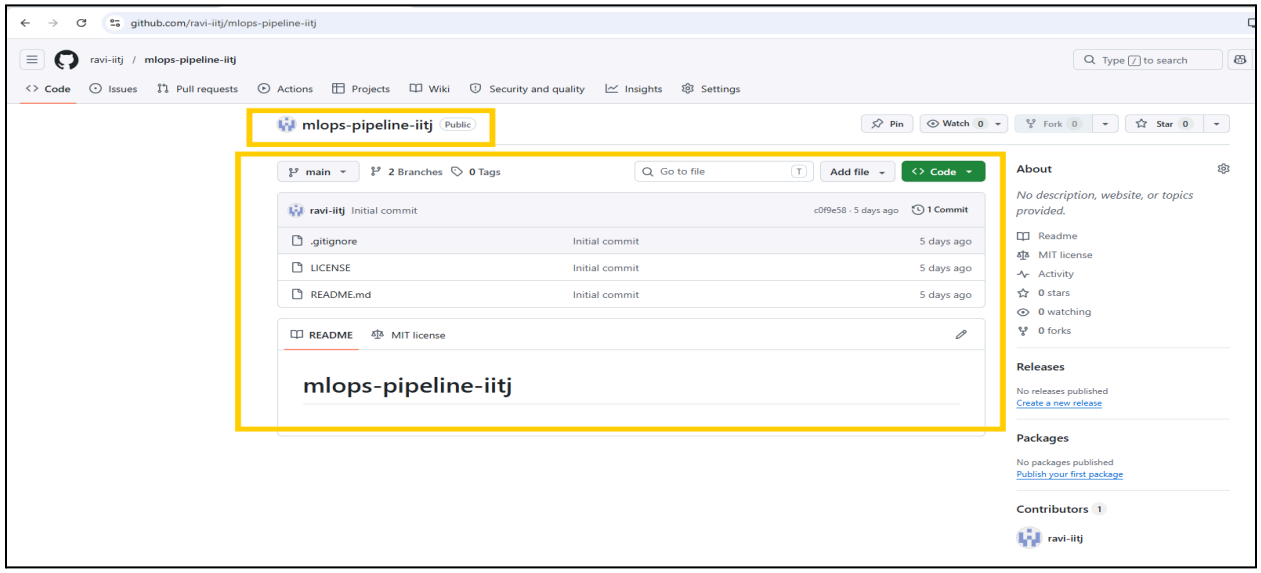
Name	Student ID	Contribution
A RAVI	**G25AIT2001**	I was responsible for setting up the project foundation. For Task 1, I created the public GitHub repository, configured the folder structure, established the develop branch, enforced main branch protection rules requiring pull request reviews, and added all teammates as collaborators with Write access. For Task 2, I wrote the complete data preparation pipeline, performed raw data inspection, applied text cleaning steps including lowercasing, HTML removal and punctuation stripping, removed duplicate entries, created the id2label.json label mapping file, and saved the cleaned datasets.
Shrijaya Chauhan	**G25AIT2146**	contributed to Task 6 and Task 7 of the MLOps project. For Task 6, containerized the fine-tuned DistilBERT sentiment analysis model using Docker by creating the Dockerfile, inference script, and requirements file, building the Docker image, validating inference execution, and publishing the image to Docker Hub. For Task 7,implemented GitHub Actions workflows for CI/CD automation, including the CI workflow (ci.yml) and inference workflow (inference.yml), verified successful workflow execution, and documented the complete process with screenshots and results in the final project report (Tasks 6–7 (Dockerization, Docker Hub deployment, GitHub Actions CI/CD workflows).
Pinaki Jayakar	**G25AIT2045**	I contributed to Task 3, Task 4, and Task 5 of the MLOps project. For Task 3, selected and configured the DistilBERT (distilbert-base-uncased) model for sentiment classification, loaded the tokenizer and classification model, and documented the model selection rationale. For Task 4, fine-tuned the model on Kaggle using the Hugging Face Trainer API, conducted two experiments with different learning

		rates, integrated Weights & Biases (W&B) for experiment tracking, logged training metrics and hyperparameters, and compared both versions to identify the best-performing model. For Task 5, pushed the trained model weights and tokenizer to the Hugging Face Hub, logged the model URL in the W&B run summary, and documented the complete process in the final project report.

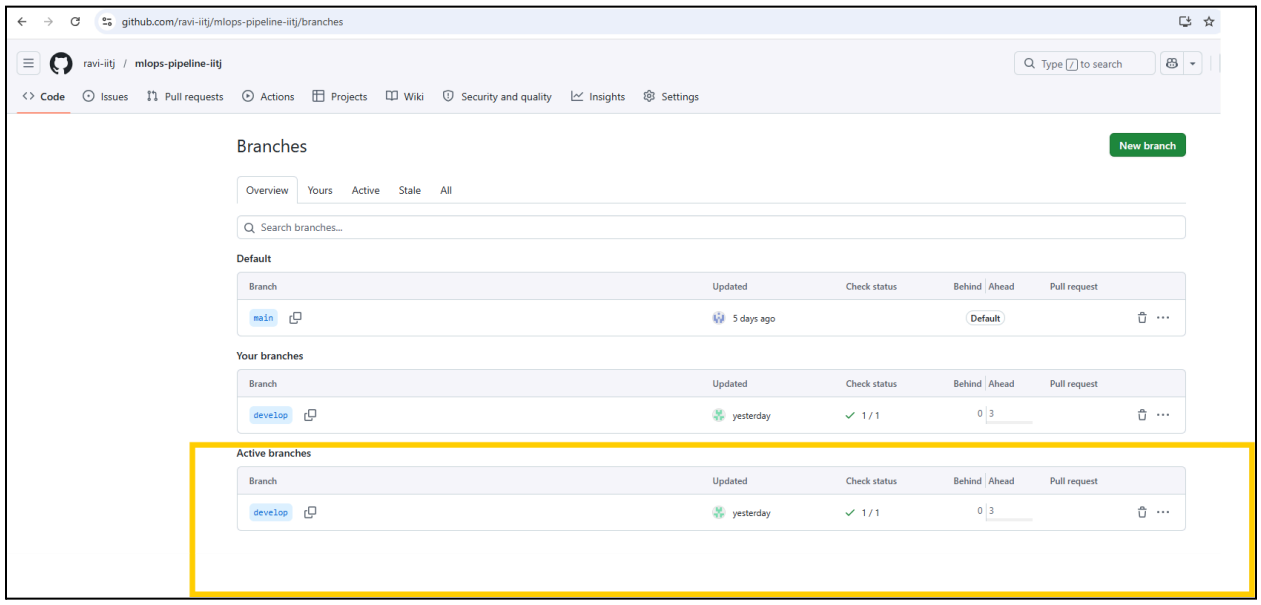
Task 1:

Github Repository Link : https://github.com/ravi-iitj/mlops-pipeline-iitj

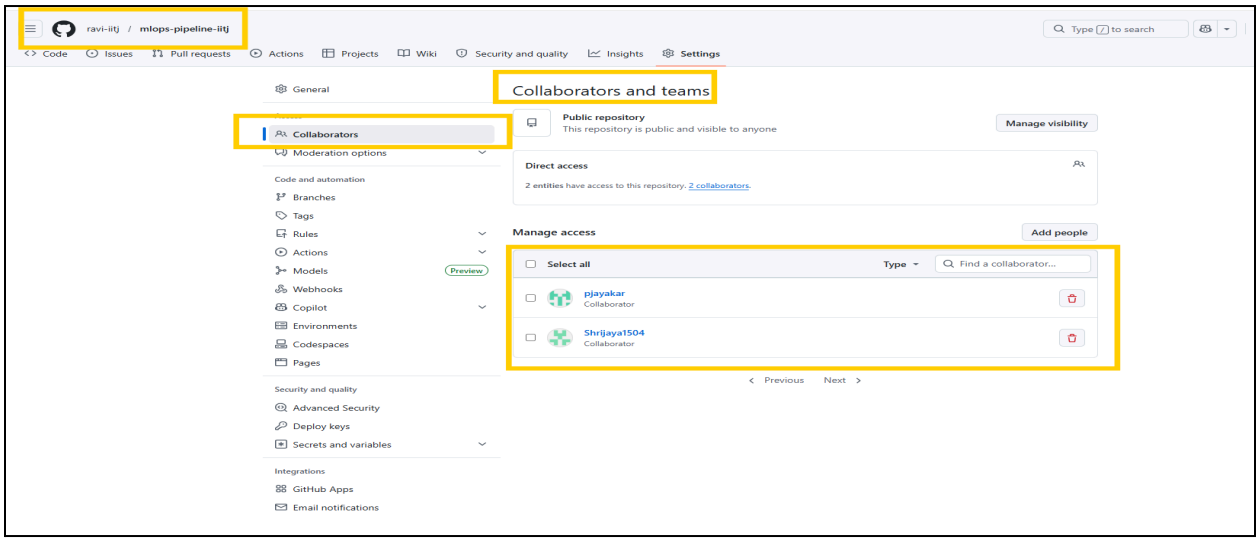
- new public GitHub repository with a README.md, .gitignore, and LICENSE



- Add a develop branch; protect main (require at least 1 pull-request review before merging)



- One team member is the **Admin** (repository owner); all others are added as **Collaborators** with Write access



- Include a screenshot of the Collaborators settings page in your report

Task 2 :
Kaggle URL :

https://www.kaggle.com/code/raviaiethagoni/ml-ops-data-preperation

Dataset Selected
IMDb Sentiment Dataset (Hugging Face) — 25,000 train + 25,000 test movie reviews for binary sentiment classification (positive / negative)

Missing Values

Inspected both splits using isnull().sum(). Found **zero missing values** across all 50,000 rows. Applied dropna() defensively to ensure pipeline robustness for future data sources.

Text Cleaning Steps

Step	Process	Reason
Lowercasing	Converted all text to lowercase	Sentiment is not case-sensitive; reduces vocabulary size
Whitespace Normalisation	Collapsed multiple spaces into one	Ensures clean, uniform input after previous cleaning steps
Punctuation Removal	Stripped all non-alphanumeric characters	Reduces tokenizer noise and vocabulary size

Duplicate Removal

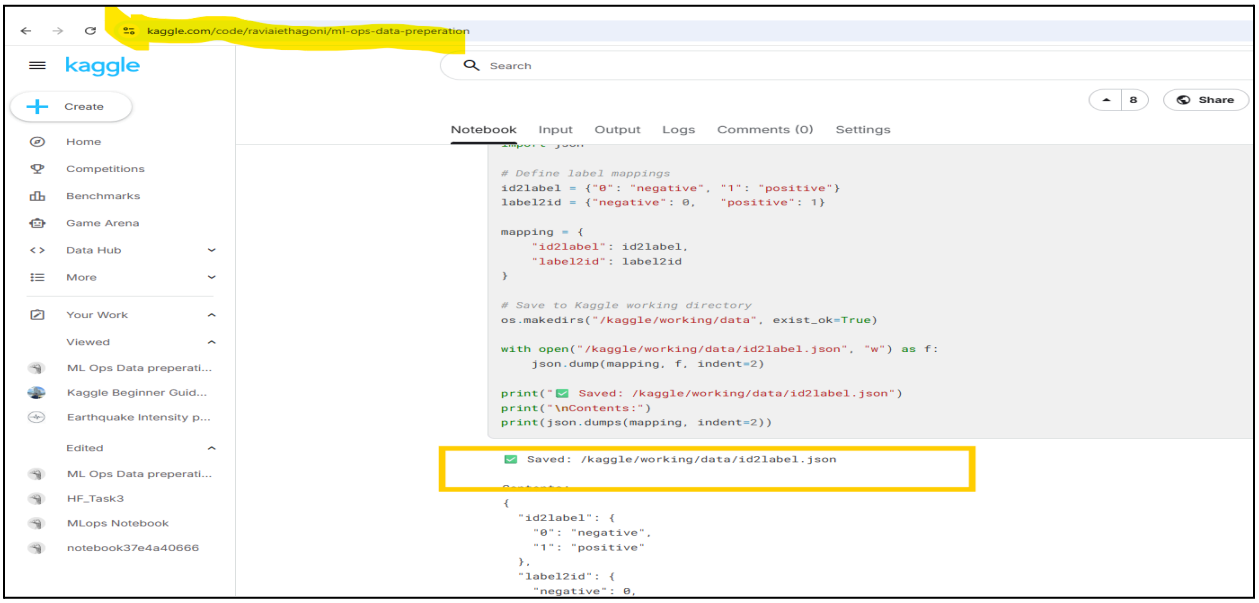
Split	**Before**	**After**	**Removed**
Train	25000	**24997**	3 Duplicates
Test	25000	**2500**	**0**

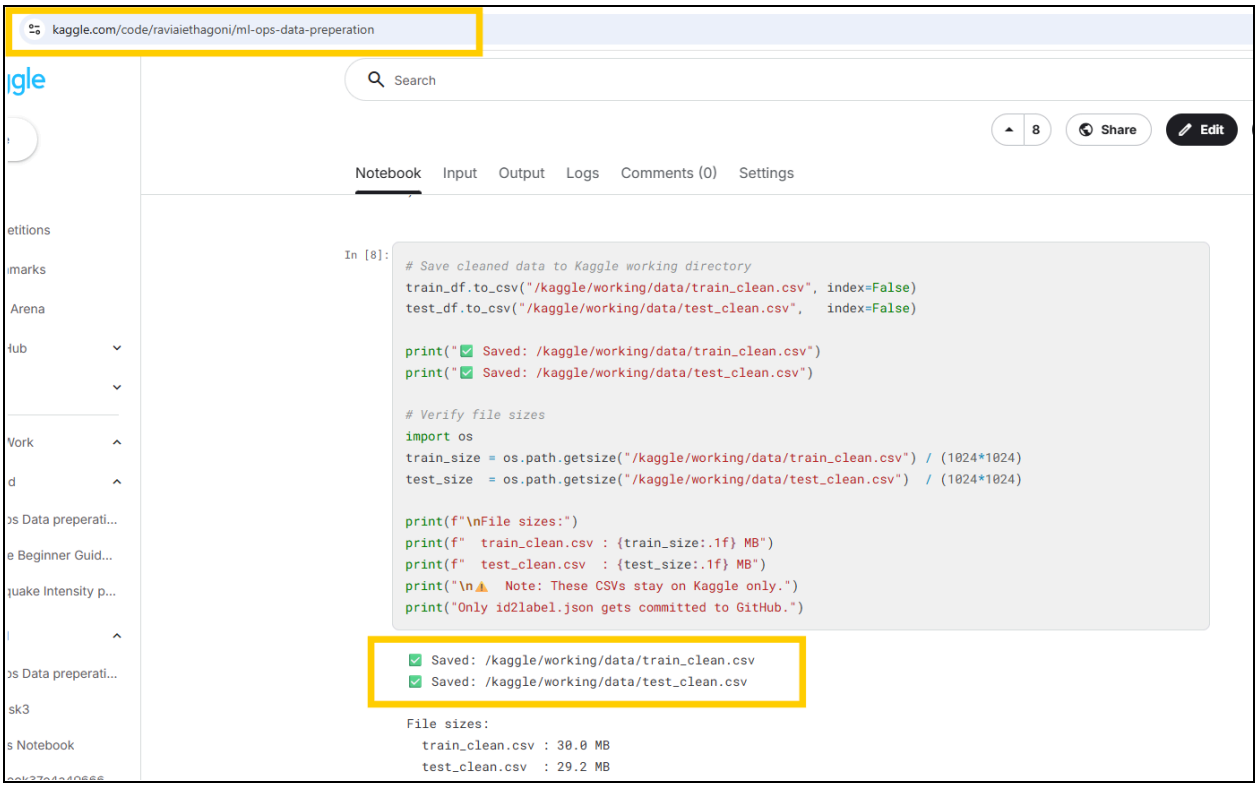
Class Distribution

Lable	**Class**	**Class**	**Percentage(%)**
0	Negative	12498	50
1	Positive	12497	50

Normalisation Strategy

Applied text normalisation (not numeric scaling since this is a text task). Stopwords and stemming were intentionally skipped as DistilBERT handles context and word variants natively through its WordPiece tokenizer.





Task 3:
Model Selection Rationale

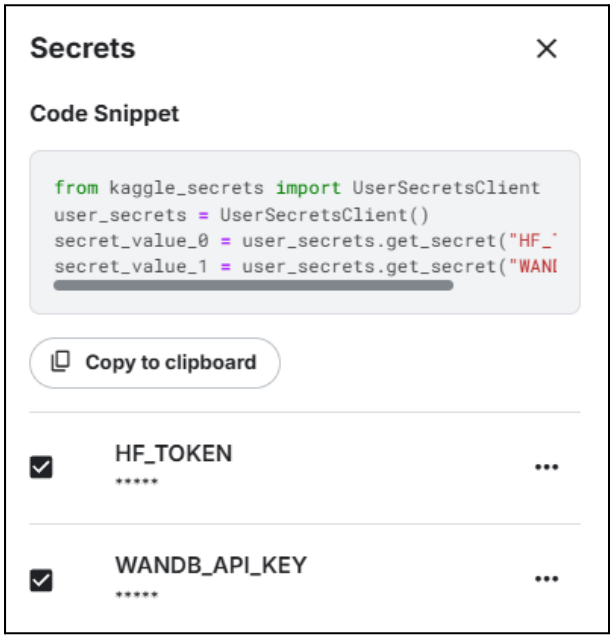
We went with distilbert-base-uncased for this sentiment classification task primarily because of its size and speed. DistilBERT is a lighter version of BERT. It has approximately 67 million parameters and is significantly smaller than the original BERT model, yet it retains around 97% of BERT's performance on most language tasks. This made it a practical choice given Kaggle's free GPU constraints.

The model was pre-trained on a large English corpus using masked language modelling combined with a knowledge distillation objective, meaning it learned from BERT's outputs rather than raw data alone. This gives it a strong contextual understanding of text right out of the box, which matters a lot for sentiment tasks where phrasing and context are everything.

According to the Hugging Face model card, the base variant is task-agnostic, so fine-tuning it on IMDb with a two-label classification head (negative / positive) is a natural fit. We attached the head by setting num_labels=2 with our id2label mapping, and the model adapted cleanly to the binary task.

Task 4:

The model was fine-tuned using the Hugging Face Trainer API on a Kaggle Notebook with GPU acceleration enabled. Sensitive credentials such as the Hugging Face token and W&B API key were securely managed using Kaggle Secrets instead of being hardcoded.



Two different training experiments were conducted by modifying the hyperparameters to compare model performance. All experiments were tracked using Weights & Biases (W&B), where training loss, validation loss, accuracy, F1 score, and training configurations were automatically logged.

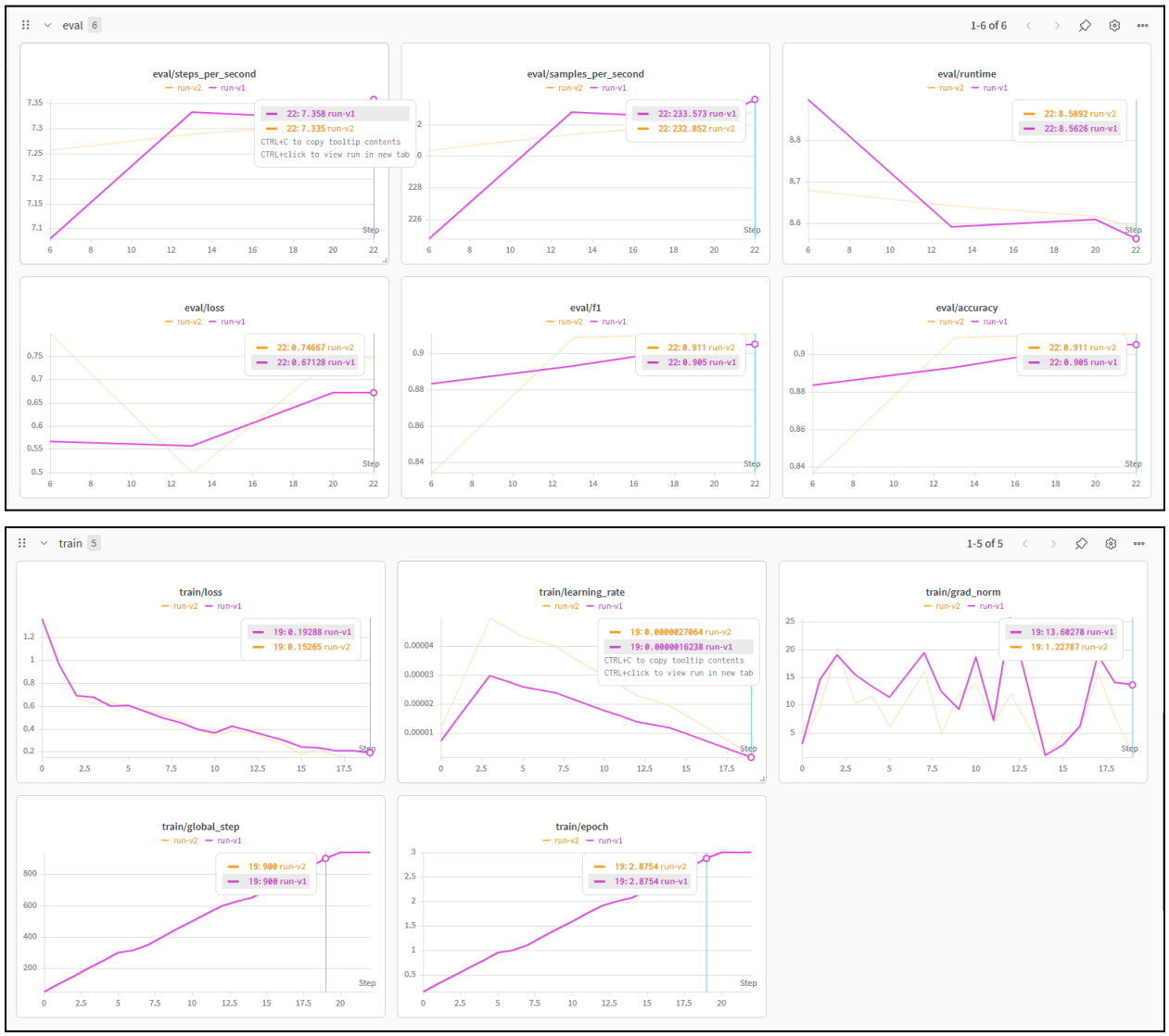
Experiment Comparison Results:

Metric	Version 1	Version 2
Learning Rate	3e-5	5e-5
Accuracy	0.9050	0.9110
F1 Score	0.9050	0.9110
Validation Loss	0.6713	0.7467

Version 2 achieved the highest accuracy (91.10%) and weighted F1 score (91.10%), outperforming Version 1. Although Version 2 recorded a slightly higher validation loss, its

superior classification performance made it the preferred model for deployment. The comparison available through the W&B dashboard helped in selecting the best-performing model.

Weights and Biases Dashboard



Task 5:

The trained model weights and tokenizer were successfully pushed to the Hugging Face Hub, and the model repository URL was logged to the W&B run summary for experiment traceability.

Links

W&B	https://wandb.ai/pinakijayakar01-iitj/mlops-assignment3
HF	https://huggingface.co/pjayG25AIT2045/distilbert-imdb-mlops
GitHub	https://github.com/ravi-iitj/mlops-pipeline-iitj
Kaggle	https://www.kaggle.com/code/raviaiethagoni/ml-ops-data-preperation

Task 6: **Docker Containerization**

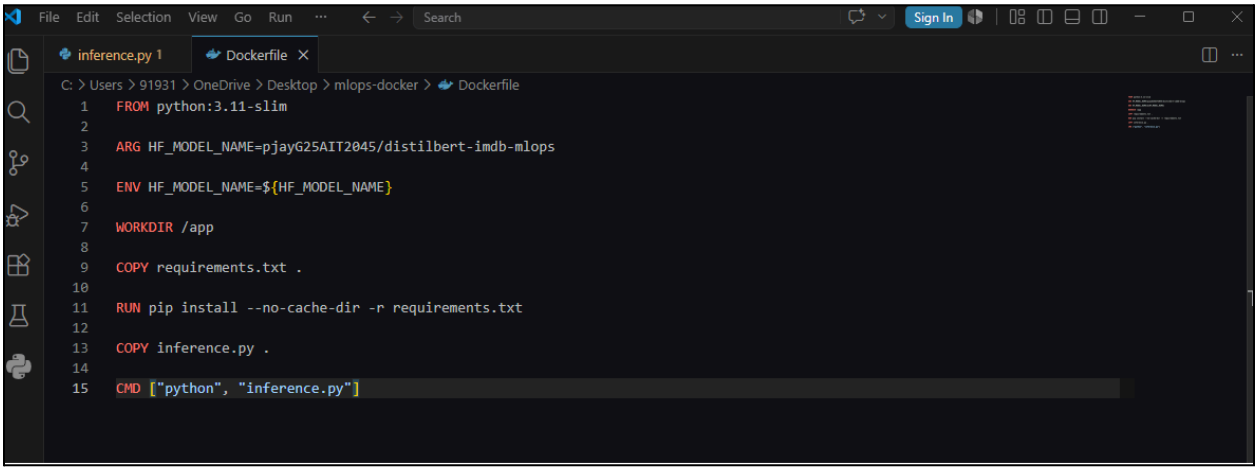
The fine-tuned DistilBERT sentiment analysis model was containerized using Docker to ensure portability and reproducibility across environments.

Steps Performed:
• Created an inference.py script to load the Hugging Face model and perform sentiment prediction.
• Created a Dockerfile based on the Python 3.11 slim image.
• Installed required dependencies using requirements.txt.
• Built the Docker image locally using Docker Desktop.
• Successfully executed the container and verified inference output.
• Pushed the Docker image to Docker Hub for public access.

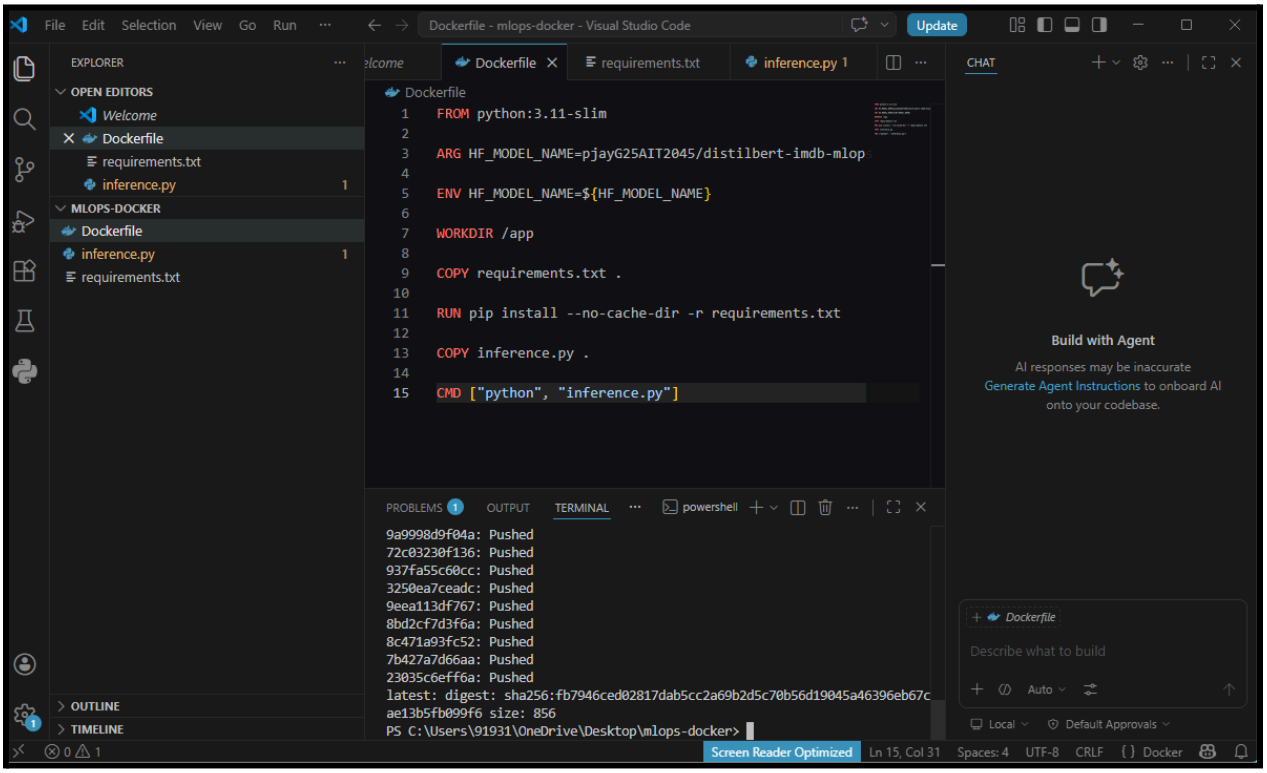
Docker Hub Repository:
https://hub.docker.com/r/shrijaya2146/mlops-a3-inference

Sample Inference Output:
Prediction: POSITIVE
Confidence Score: 0.996

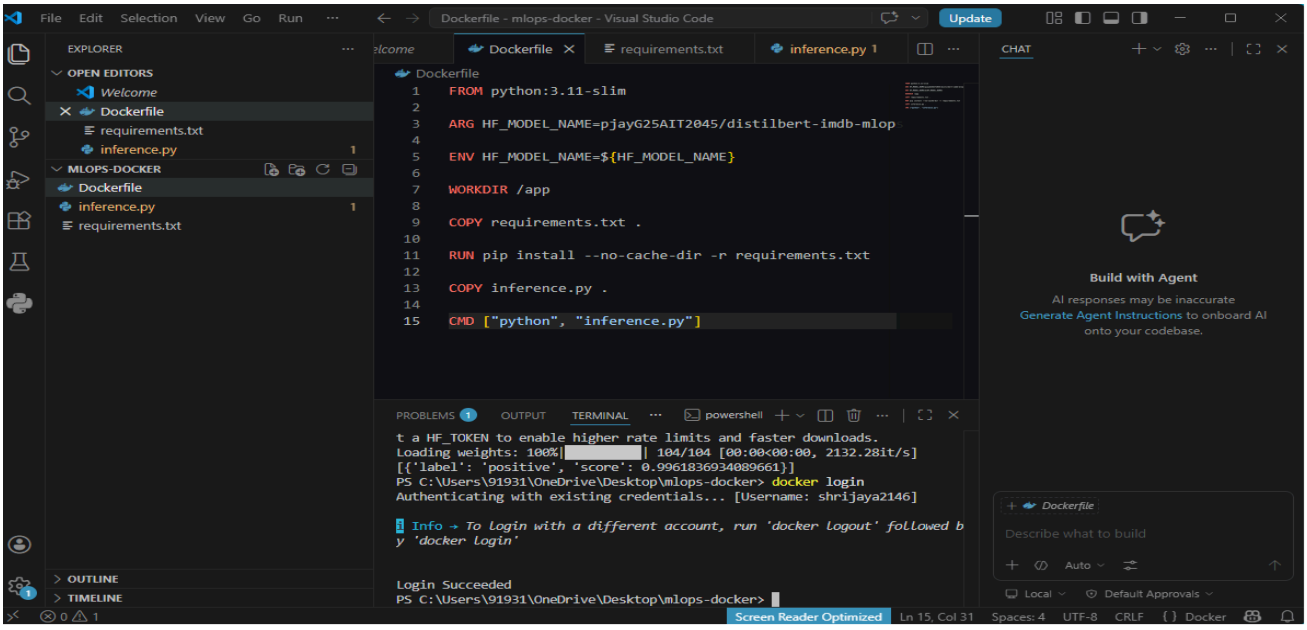
Outcome:
The trained model was successfully packaged into a reusable Docker container and made available through Docker Hub.



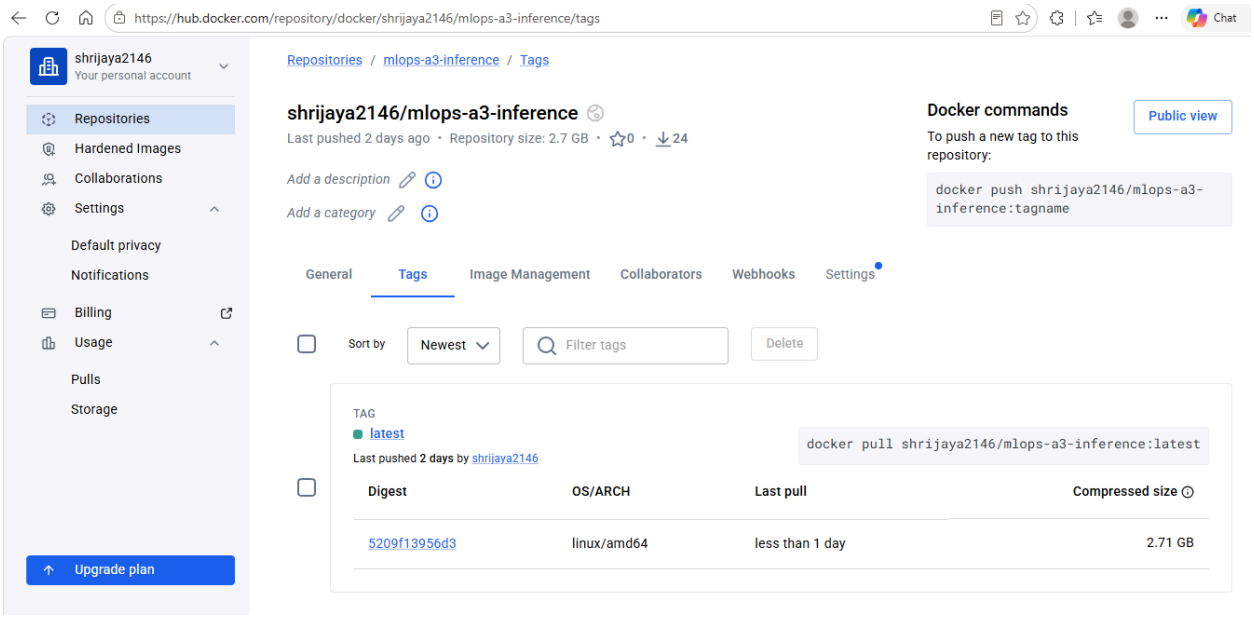
- Dockerfile used to package the DistilBERT sentiment analysis model into a Docker container. It defines the Python base image, installs dependencies, copies the inference script, and specifies the container execution command.



- Successful Docker image build.



- Successful execution of Docker container and inference output.



- **Docker Image Published on Docker Hub** The Docker image containing the DistilBERT sentiment analysis model was successfully pushed to Docker Hub and made available through the public repository

Task 7: CI/CD Automation using GitHub Actions

To automate model validation and deployment workflows, GitHub Actions was integrated into the project repository.

Implemented Workflows:

CI Workflow (ci.yml)
• Triggered on code pushes to the develop branch.
• Validates repository structure and workflow execution.

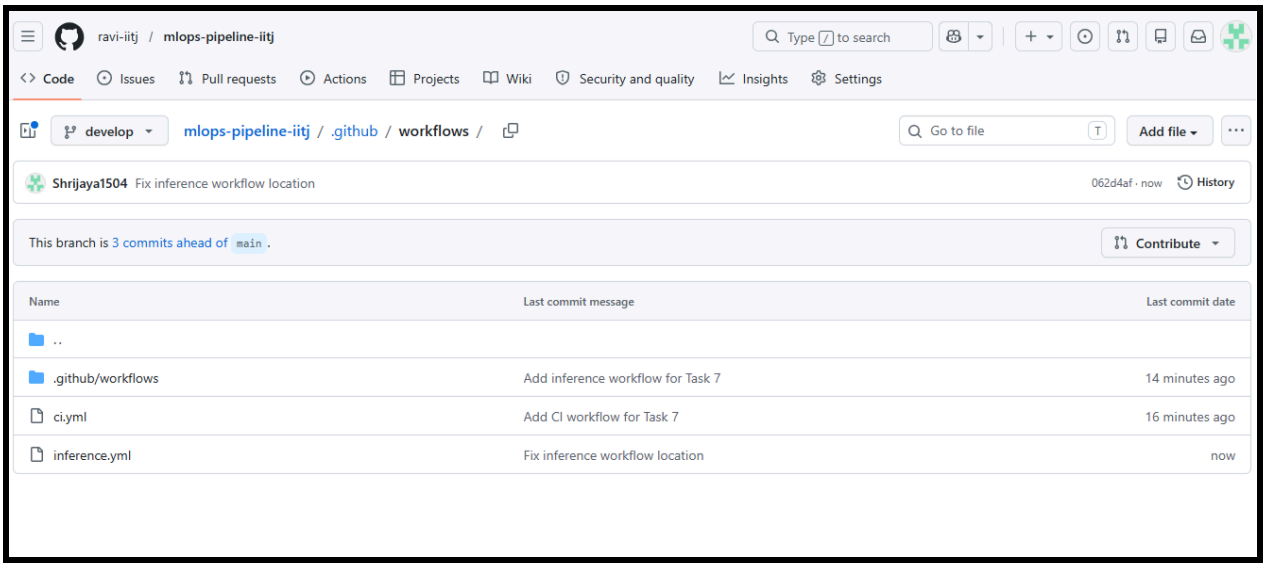
Inference Workflow (inference.yml)
• Pulls the Docker image from Docker Hub.
• Executes the container automatically.
• Verifies that inference runs successfully inside the GitHub Actions environment.

Results:
• Both workflows executed successfully on the develop branch.
• Workflow runs completed without errors.
• All commits were tracked through GitHub Actions logs.

Outcome:
The project now supports basic CI/CD automation, ensuring repeatable validation and deployment of the sentiment analysis pipeline .

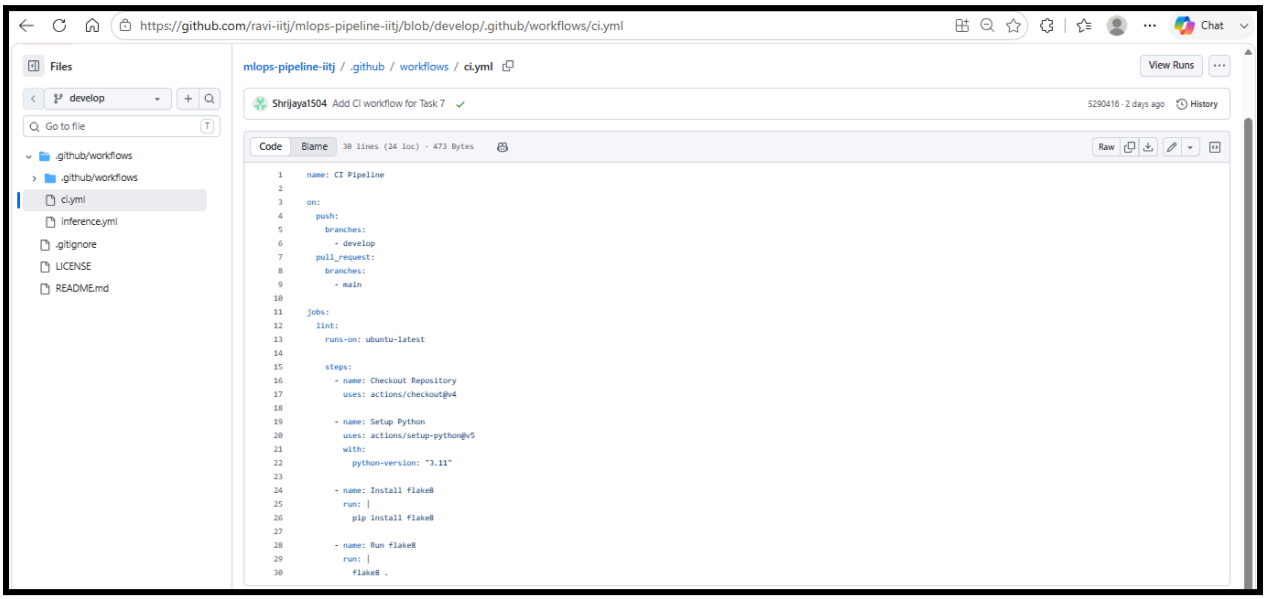
GitHub Repository: https://github.com/ravi-iitj/mlops-pipeline-iitj

Docker Hub Repository:https://hub.docker.com/r/shrijaya2146/mlops-a3-inference



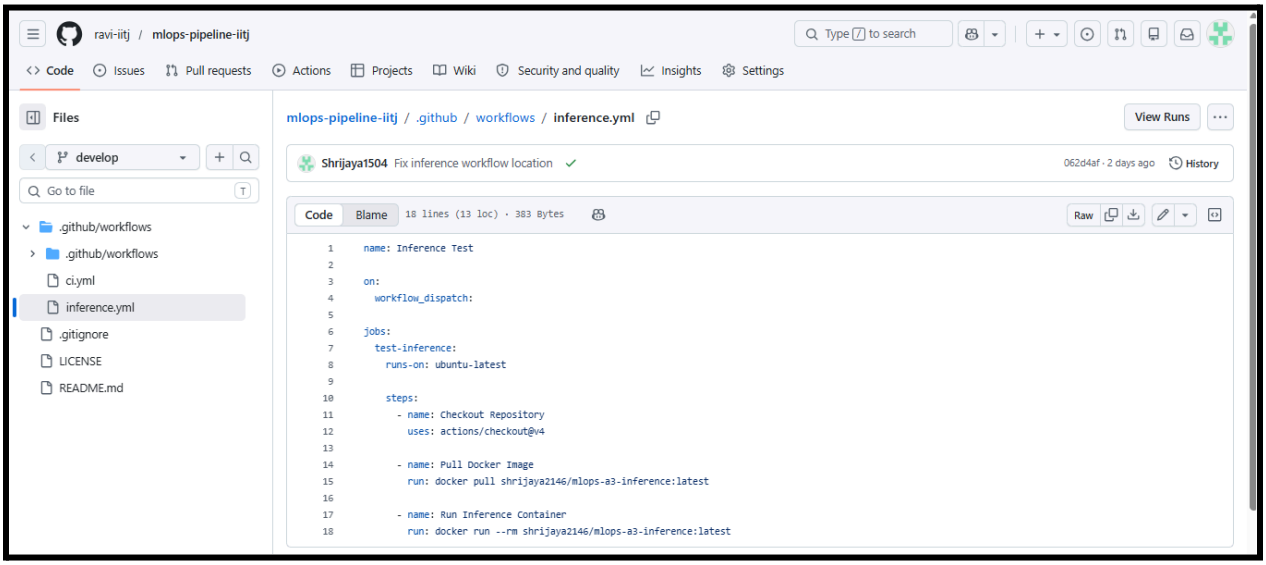
- **GitHub Actions workflow files**

The GitHub repository contains two workflow files (ci.yml and inference.yml) used to automate CI/CD operations and Docker-based inference testing.



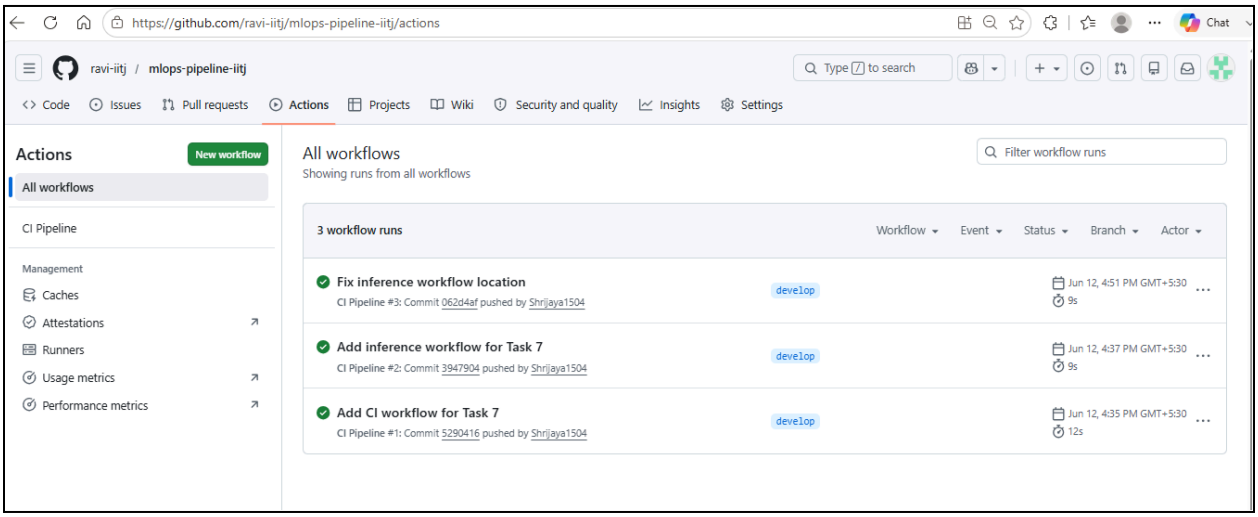
- **CI Workflow Configuration (ci.yml)**

GitHub Actions CI workflow configured to automatically validate code quality on pushes to the develop branch and pull requests to the main branch.



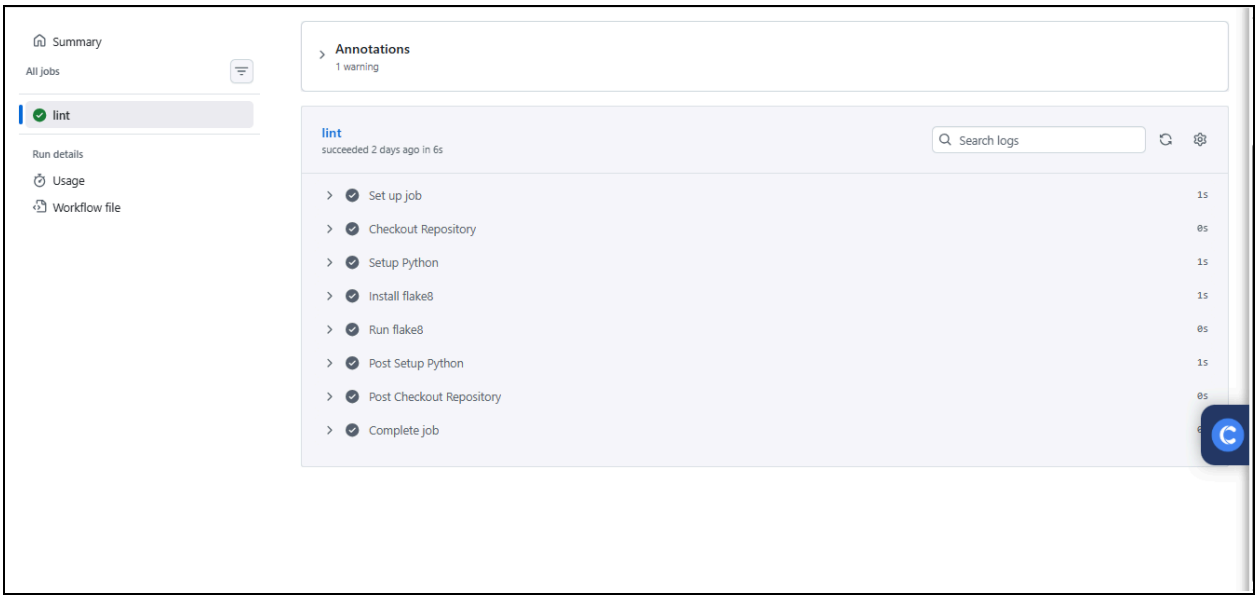
- **Inference Workflow Configuration (inference.yml)**

GitHub Actions workflow configured to automatically pull the Docker image from Docker Hub and execute the sentiment analysis inference container.



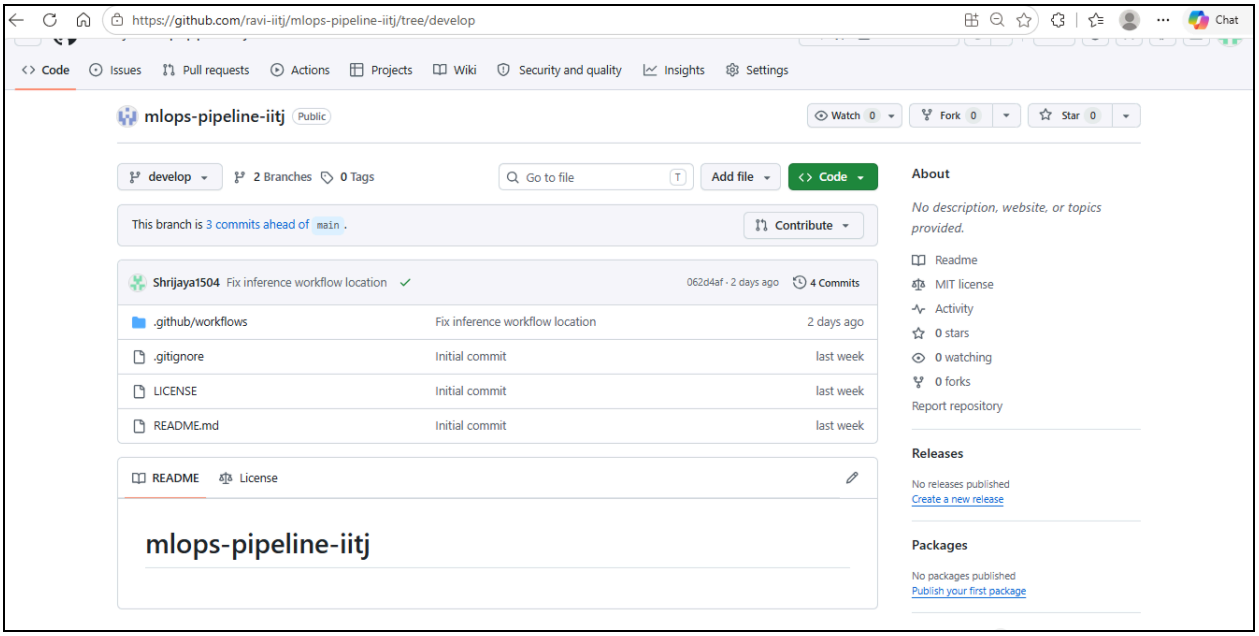
- **Successful GitHub Actions Workflow Execution**

GitHub Actions successfully executed the CI and inference workflows. All workflow runs completed without errors, validating the automated CI/CD pipeline.



- **Successful CI Pipeline Execution in GitHub Actions**

The GitHub Actions CI workflow executed successfully on the develop branch. The workflow checked out the repository, configured Python, installed flake8, performed code quality checks, and completed without errors.



- **Repository Commit History and Workflow Integration**

The repository shows successful integration of CI/CD workflow files in the develop branch. The branch is three commits ahead of main, reflecting the addition and refinement of GitHub Actions workflows for automated testing and inference validation.

Challenges Faced

Task 1 – GitHub Repository Setup (Ravi)

- Creating and organizing the repository structure.
- Managing branches and collaborator access.
- Ensuring all team members could contribute through GitHub.

Task 2 – Dataset Analysis (Ravi)

- Understanding dataset characteristics and preprocessing requirements.
- Identifying missing values, duplicates, and text-cleaning needs.

Tasks 3–5 – Model Training, W&B and Hugging Face (Pinaki)

- Selecting an appropriate pre-trained model for sentiment analysis.
- Comparing multiple experiment runs and tracking metrics using Weights & Biases.
- Uploading and versioning the trained model on Hugging Face Hub.

Task 6 – Dockerization (Shrijaya)

- Creating a Docker image with all required dependencies.
- Ensuring the Hugging Face model loaded correctly inside the container.
- Managing large image size and successful Docker Hub publishing.

Task 7 – CI/CD Automation (Shrijaya)

- Understanding GitHub Actions workflow syntax.
- Debugging workflow file locations and execution issues.
- Verifying successful workflow execution through GitHub Actions.

Key Learnings

- Learned Git and GitHub collaboration using branches, commits, and workflow management.
- Gained experience in dataset preprocessing and sentiment analysis workflows.
- Understood experiment tracking and model versioning using Weights & Biases.
- Learned how to publish and share machine learning models through Hugging Face Hub.
- Acquired practical experience in Docker containerization and image management.
- Implemented CI/CD automation using GitHub Actions for machine learning projects.
- Understood how different MLOps tools integrate to create an end-to-end machine learning pipeline

Conclusion

This project successfully implemented an end-to-end MLOps pipeline for sentiment analysis using the IMDb dataset. The workflow included data preparation, model training, experiment tracking, model hosting on Hugging Face, containerization using Docker, and CI/CD automation through GitHub Actions. The final solution ensures reproducibility, portability, and automated validation of the machine learning workflow, demonstrating key MLOps principles in practice.